서울시 상권 유형별 소비 패턴 군집화 분석

- 26-1 기계학습 HW2
- 정보융합학부 김민지 (2023510005)
- 서울 열린데이터광장의 상권분석서비스 데이터를 활용해 서울시 상권을 군집화하고, 실제 상권 구분(골목상권 / 발달상권 / 전통시장 / 관광특구)과 비교 분석한 프로젝트
- 데이터 출처: 서울 열린데이터광장

> 상권분석서비스(추정매출-상권)
> 상권분석서비스(점포-상권)
> 상권분석서비스(직장인구-상권)

분석 단위: 서울시 상권 (1,581개)
Label (y): 상권_구분_코드_명 (골목상권 / 발달상권 / 전통시장 / 관광특구)

0. 라이브러리 임포트

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
import matplotlib
import warnings
warnings.filterwarnings('ignore')

# 한글 폰트 설정
def set_korean_font():
    # 폰트 목록 (Mac / Linux / Windows 순서)
    candidates = [
        'AppleGothic',          # Mac
        'Apple SD Gothic Neo',  # Mac
        'NanumGothic',          # Linux/공통
        'NanumBarunGothic',
        'Malgun Gothic',        # Windows
        'DejaVu Sans',          # 폴백
    ]
    available = {f.name for f in fm.fontManager.ttflist}
    for font in candidates:
        if font in available:
            matplotlib.rcParams['font.family'] = font
            matplotlib.rcParams['axes.unicode_minus'] = False
            print(f'폰트 설정 완료: {font}')
            return font
    print('한글 폰트를 찾지 못했습니다. 아래 나눔폰트 설치를 권장합니다.')
    return None
```

```
set_korean_font()

import seaborn as sns

from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.cluster import KMeans, AgglomerativeClustering,
DBSCAN, AffinityPropagation
from sklearn.metrics import silhouette_score, silhouette_samples
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE

from scipy.cluster.hierarchy import dendrogram, linkage
from scipy.spatial.distance import cdist

print('라이브러리 로드 완료')
```

```
폰트 설정 완료: AppleGothic
라이브러리 로드 완료
```

---

1. 데이터 수집 및 이해

1-1. 데이터 로드

In [2]:
```
# 파일 경로
base_path = '/Users/gomdori/Library/Mobile
Documents/com~apple~CloudDocs/26-1 기계학습/ML_HW2_seoul-commercial-
district-clustering'
SALES_PATH = f'{base_path}/Data/서울시_상권분석서비스(추정매출-상권)_2024
년.csv'
STORE_PATH = f'{base_path}/Data/서울시_상권분석서비스(점포-상권)_2024
년.csv'
WORK_PATH  = f'{base_path}/Data/서울시_상권분석서비스(직장인구-상권)_2024
년.csv'
```

추정매출-상권

In [3]:
```
df_sales = pd.read_csv(SALES_PATH, encoding='utf-8')
print(f'추정매출 shape: {df_sales.shape}')
df_sales.head(5)
```

```
추정매출 shape: (87179, 55)
```

Out[3]:

	기준_년분기_코드	상권_구분_코드	상권_구분_코드_명	상권_코드	상권_코드_명	서비스_업종_코드	서비스_업종_코드_명	당월_매출_금액	당월_매출_건수	주중_매출
0	20241	A	골목상권	3110001	이북5도청사	CS100001	한식음식점	385150643.0	11743	2201292
1	20241	A	골목상권	3110001	이북5도청사	CS100009	호프-간이주점	11026440.0	214	65762
2	20241	A	골목상권	3110001	이북5도청사	CS200001	일반교습학원	52047765.0	224	505352
3	20241	A	골목상권	3110002	독립문역1번	CS100001	한식음식점	582522265.0	20630	4045094
4	20241	A	골목상권	3110002	독립문역1번	CS100005	제과점	446525280.0	40242	3502647

5 rows × 55 columns

점포-상권

In [4]:

```
df_store = pd.read_csv(STORE_PATH, encoding='utf-8')
print(f'점포 shape: {df_store.shape}')
df_store.head(5)
```

```
점포 shape: (306889, 14)
```

Out[4]:

	기준_년분기_코드	상권_구분_코드	상권_구분_코드_명	상권_코드	상권_코드_명	서비스_업종_코드	서비스_업종_코드_명	점포_수	유사_업종_점포_수	개업_율	개업_점포_수	폐업_률	폐업_점포_수	프랜차이즈_점포_수
0	20241	A	골목상권	3110001	이북5도청사	CS100001	한식음식점	10	11	0	0	0	0	1
1	20241	A	골목상권	3110001	이북5도청사	CS100003	일식음식점	1	1	0	0	0	0	0
2	20241	A	골목상권	3110001	이북5도청사	CS100008	분식전문점	2	2	0	0	50	1	0
3	20241	A	골목상권	3110001	이북5도청사	CS100009	호프-간이주점	4	5	20	1	0	0	1
4	20241	A	골목상권	3110001	이북5도청사	CS100010	커피-음료	1	1	0	0	0	0	0

직장인구-상권

In [5]:

```
df_work  = pd.read_csv(WORK_PATH,  encoding='utf-8')
print(f'직장인구 shape: {df_work.shape}')
df_work.head(5)
```

```
직장인구 shape: (45840, 26)
```

Out[5]:

	기준_	상권_	상권_구	상권_	총_직장	남성_직	여성_	연령대_10_	연령대_20_	남

	년분기_코드	구분_코드	분_코드_명	상권_코드	코드_명	_인구_수	장_인구_수	직장_인구_수	직장_인구_수	직장_인구_수	...	
0	20254	U	관광특구	3001496	강남마이스관광특구	102032	68834	33198	260	20558	...	20
1	20254	U	관광특구	3001495	잠실관광특구	137602	81307	56295	247	22732	...	22
2	20254	U	관광특구	3001494	종로?청계관광특구	46516	26098	20418	658	9450	...	7
3	20254	U	관광특구	3001493	동대문패션타운관광특구	33754	16984	16770	259	9770	...	5
4	20254	U	관광특구	3001492	명동남대문북창동다동무교동관광특	199667	114485	85182	372	40531	...	35

구

5 rows × 26 columns

1-2. 데이터 이해

[추정매출-상권] 55개 컬럼, 87,179행 (상권 × 업종 단위)

- 당월 매출금액/건수, 주중·주말, 요일별(7), 시간대별(6), 성별(2), 연령대별(6) 매출

[점포-상권] 14개 컬럼, 306,889행 (상권 × 업종 단위)

- 점포수, 유사업종점포수, 개업률·개업점포수, 폐업률·폐업점포수, 프랜차이즈점포수

[직장인구-상권] 26개 컬럼, 45,840행 (상권 × 분기 단위)

- 총직장인구, 남/여, 연령대별(6), 성별×연령대별(12) 직장인구

분석 전략: 2024년 4분기(20244) 기준으로 필터 후, 상권 단위로 집계하여 1,581개 상권을 관측치로 분석

In [6]:
```
# 분기별 분포 확인
TARGET_Q = 20244  # 2024년 4분기

print('[상권 구분 분포]')
print(df_sales[df_sales['기준_년분기_코드'] == TARGET_Q]
      .groupby('상권_구분_코드_명')['상권_코드'].nunique())

df_sales.head(3)
```

```
[상권 구분 분포]
상권_구분_코드_명
골목상권     1035
관광특구        6
발달상권      249
전통시장      280
Name: 상권_코드, dtype: int64
```

Out[6]:

	기준_년분기_코드	상권_구분_코드	상권_구분_코드_명	상권_코드	상권_코드_명	서비스_업종_코드	서비스_업종_코드_명	당월_매출_금액	당월_매출_건수	주중_매출_
0	20241	A	골목상권	3110001	이북5도청사	CS100001	한식음식점	385150643.0	11743	220129223
1	20241	A	골목상권	3110001	이북5도청사	CS100009	호프-간이주점	11026440.0	214	6576206
2	20241	A	골목상권	3110001	이북5도청사	CS200001	일반교습학원	52047765.0	224	50535211

3 rows × 55 columns

2. 전처리

2-1. 분기 필터 및 상권 단위 집계

In [7]:

```
# ── 추정매출: 2024Q4 필터 후 상권별 sum ──────────────────────────
sales_num_cols = [
    '당월_매출_금액', '당월_매출_건수',
    '주중_매출_금액', '주말_매출_금액',
    '월요일_매출_금액', '화요일_매출_금액', '수요일_매출_금액',
    '목요일_매출_금액', '금요일_매출_금액', '토요일_매출_금액', '일요일_매출_금
액',
    '시간대_00~06_매출_금액', '시간대_06~11_매출_금액', '시간대_11~14_매출_금
액',
    '시간대_14~17_매출_금액', '시간대_17~21_매출_금액', '시간대_21~24_매출_금
액',
    '남성_매출_금액', '여성_매출_금액',
    '연령대_10_매출_금액', '연령대_20_매출_금액', '연령대_30_매출_금액',
```

```
    '연령대_40_매출_금액', '연령대_50_매출_금액', '연령대_60_이상_매출_금액'
]

sales_q = df_sales[df_sales['기준_년분기_코드'] == TARGET_Q].copy()
sales_agg = (
    sales_q.groupby(['상권_코드', '상권_코드_명', '상권_구분_코드_명'])
[sales_num_cols]
    .sum()
    .reset_index()
)

# ── 점포: 2024Q4 필터 후 상권별 sum ──────────────────────────────
store_num_cols = ['점포_수', '유사_업종_점포_수', '개업_점포_수', '폐업_점포_
수', '프랜차이즈_점포_수']

store_q = df_store[df_store['기준_년분기_코드'] == TARGET_Q].copy()
store_agg = (
    store_q.groupby('상권_코드')[store_num_cols]
    .sum()
    .reset_index()
)

# ── 직장인구: 2024Q4 필터 ────────────────────────────────────────
work_num_cols = [
    '총_직장_인구_수', '남성_직장_인구_수', '여성_직장_인구_수',
    '연령대_10_직장_인구_수', '연령대_20_직장_인구_수', '연령대_30_직장_인구_
수',
    '연령대_40_직장_인구_수', '연령대_50_직장_인구_수', '연령대_60_이상_직장_인
구_수'
]

work_q = df_work[df_work['기준_년분기_코드'] == TARGET_Q][['상권_코드']
+ work_num_cols].copy()

print(f'매출 집계: {sales_agg.shape}, 점포 집계: {store_agg.shape}, 직장
인구: {work_q.shape}')
```

```
매출 집계: (1570, 28), 점포 집계: (1650, 6), 직장인구: (1641, 10)
```

2-2. 비율 변수 생성 (상권 규모 정규화)

In [8]:

```
# 절대금액 대신 '비율 변수'를 사용 → 상권 크기 편차 제거
eps = 1  # 0 나누기 방지

# 주중/주말 비율
sales_agg['주중_매출_비율'] = sales_agg['주중_매출_금액'] /
(sales_agg['당월_매출_금액'] + eps)
sales_agg['주말_매출_비율'] = sales_agg['주말_매출_금액'] /
(sales_agg['당월_매출_금액'] + eps)

# 요일별 비율
for day in ['월요일', '화요일', '수요일', '목요일', '금요일', '토요일', '일요
일']:
    sales_agg[f'{day}_매출_비율'] = sales_agg[f'{day}_매출_금액'] /
(sales_agg['당월_매출_금액'] + eps)
```

```
# 시간대별 비율
for t in ['00~06', '06~11', '11~14', '14~17', '17~21', '21~24']:
    sales_agg[f'시간대_{t}_비율'] = sales_agg[f'시간대_{t}_매출_금액'] /
(sales_agg['당월_매출_금액'] + eps)

# 성별 비율
sales_agg['여성_매출_비율'] = sales_agg['여성_매출_금액'] /
(sales_agg['당월_매출_금액'] + eps)

# 연령대별 비율
for age in ['10', '20', '30', '40', '50', '60_이상']:
    sales_agg[f'연령대_{age}_비율'] = sales_agg[f'연령대_{age}_매출_금
액'] / (sales_agg['당월_매출_금액'] + eps)

# 프랜차이즈 비율 (점포 대비)
store_agg['프랜차이즈_비율'] = store_agg['프랜차이즈_점포_수'] /
(store_agg['점포_수'] + eps)
store_agg['개업_비율']       = store_agg['개업_점포_수'] /
(store_agg['점포_수'] + eps)
store_agg['폐업_비율']       = store_agg['폐업_점포_수'] /
(store_agg['점포_수'] + eps)

# 직장인구 성별 비율
work_q = work_q.copy()
work_q['직장_여성_비율'] = work_q['여성_직장_인구_수'] / (work_q['총_직장_인
구_수'] + eps)
work_q['직장_2030_비율'] = (work_q['연령대_20_직장_인구_수'] + work_q['연
령대_30_직장_인구_수']) / (work_q['총_직장_인구_수'] + eps)

print('비율 변수 생성 완료')
```

```
비율 변수 생성 완료
```

2-3. 세 데이터 병합

In [9]:

```
# merge
df = sales_agg.merge(store_agg, on='상권_코드', how='left')
df = df.merge(work_q, on='상권_코드', how='left')

print(f'병합 후 shape: {df.shape}')
print(f'결측값:\n{df.isnull().sum()[df.isnull().sum() > 0]}')
```

```
병합 후 shape: (1570, 69)
결측값:
총_직장_인구_수             6
남성_직장_인구_수            6
여성_직장_인구_수            6
연령대_10_직장_인구_수        6
연령대_20_직장_인구_수        6
연령대_30_직장_인구_수        6
연령대_40_직장_인구_수        6
연령대_50_직장_인구_수        6
연령대_60_이상_직장_인구_수     6
직장_여성_비율              6
직장_2030_비율            6
dtype: int64
```

2-4. 분석용 feature 선택 및 결측치·이상치 처리

In [10]:

```
# 분석에 사용할 feature 목록 (총 28개)
FEATURE_COLS = [
    # 매출 규모 (log 변환 예정)
    '당월_매출_금액',
    # 주중/주말
    '주중_매출_비율', '주말_매출_비율',
    # 요일별 비율
    '월요일_매출_비율', '화요일_매출_비율', '수요일_매출_비율',
    '목요일_매출_비율', '금요일_매출_비율', '토요일_매출_비율', '일요일_매출_비
율',
    # 시간대별 비율
    '시간대_00~06_비율', '시간대_06~11_비율', '시간대_11~14_비율',
    '시간대_14~17_비율', '시간대_17~21_비율', '시간대_21~24_비율',
    # 성별
    '여성_매출_비율',
    # 연령대
    '연령대_10_비율', '연령대_20_비율', '연령대_30_비율',
    '연령대_40_비율', '연령대_50_비율', '연령대_60_이상_비율',
    # 점포
    '점포_수', '프랜차이즈_비율', '개업_비율', '폐업_비율',
    # 직장인구
    '총_직장_인구_수', '직장_여성_비율', '직장_2030_비율'
]

df_feat = df[['상권_코드', '상권_코드_명', '상권_구분_코드_명'] +
FEATURE_COLS].copy()

# 결측치 처리: 직장인구 없는 상권은 0으로 채움
df_feat[FEATURE_COLS] = df_feat[FEATURE_COLS].fillna(0)

# 매출 0인 상권 제거 (데이터 없는 상권)
df_feat = df_feat[df_feat['당월_매출_금액'] >
0].reset_index(drop=True)

# 매출 금액 log 변환 (우편포를 정규분포에 가깝게)
df_feat['당월_매출_금액'] = np.log1p(df_feat['당월_매출_금액'])
df_feat['점포_수']        = np.log1p(df_feat['점포_수'])
df_feat['총_직장_인구_수'] = np.log1p(df_feat['총_직장_인구_수'])
```

```
print(f'최종 분석 데이터: {df_feat.shape}')
print(f'feature 수: {len(FEATURE_COLS)}개')
df_feat[FEATURE_COLS].describe().round(3)
```

```
최종 분석 데이터: (1570, 33)
feature 수: 30개
```

Out[10]:

	당월_매출_금액	주중_매출_비율	주말_매출_비율	월요일_매출_비율	화요일_매출_비율	수요일_매출_비율	목요일_매출_비율
count	1570.000	1570.000	1570.000	1570.000	1570.000	1570.000	1570.000
mean	21.866	0.759	0.241	0.147	0.161	0.141	0.147
std	1.905	0.092	0.092	0.052	0.046	0.040	0.046
min	13.077	0.000	0.000	0.000	0.000	0.000	0.000
25%	20.813	0.710	0.195	0.124	0.142	0.126	0.130
50%	21.950	0.756	0.244	0.146	0.157	0.137	0.141
75%	23.105	0.805	0.290	0.166	0.174	0.150	0.156
max	27.841	1.000	1.000	0.877	0.813	0.526	1.000

8 rows × 30 columns

2-5. 스케일링 및 Label 분리

In [11]:

```
# Label 저장 (군집화 입력에는 사용 안 함 - external evaluation 용)
y_true_str = df_feat['상권_구분_코드_명'].values
le = LabelEncoder()
y_true = le.fit_transform(y_true_str)
print('클래스 매핑:', dict(zip(le.classes_,
le.transform(le.classes_))))

# StandardScaler 적용
scaler = StandardScaler()
X = scaler.fit_transform(df_feat[FEATURE_COLS])

print(f'\nX shape: {X.shape}')
print(f'평균(첫 3개 feature): {X.mean(axis=0)[:3].round(3)}')
print(f'표준편차(첫 3개 feature): {X.std(axis=0)[:3].round(3)}')
```

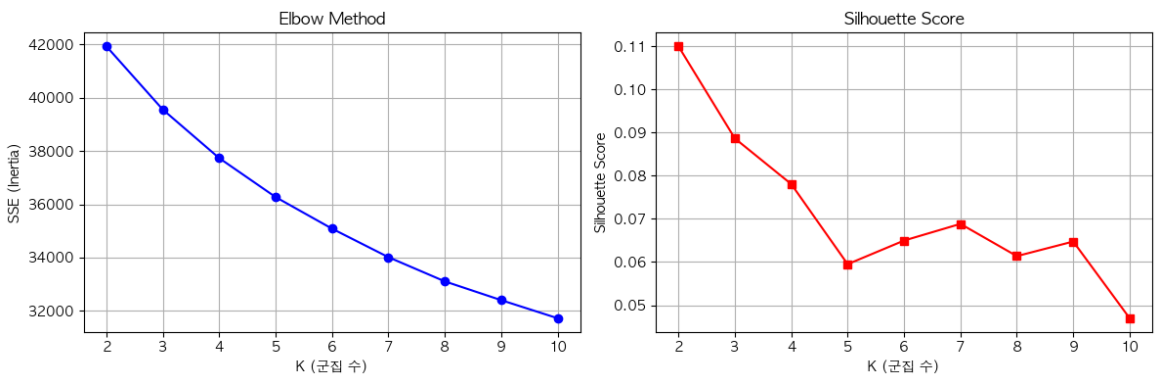
```
클래스 매핑: {'골목상권': 0, '관광특구': 1, '발달상권': 2, '전통시장': 3}

X shape: (1570, 30)
평균(첫 3개 feature): [-0.  0. -0.]
표준편차(첫 3개 feature): [1. 1. 1.]
```

3. 군집화 분석

3-1. K-Means

In [12]:

```
# ── Elbow Method (SSE) ──────────────────────────────────────────
sse = []
sil = []
K_range = range(2, 11)

for k in K_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = km.fit_predict(X)
    sse.append(km.inertia_)
    sil.append(silhouette_score(X, labels))

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].plot(K_range, sse, 'bo-')
axes[0].set_xlabel('K (군집 수)')
axes[0].set_ylabel('SSE (Inertia)')
axes[0].set_title('Elbow Method')
axes[0].grid(True)

axes[1].plot(K_range, sil, 'rs-')
axes[1].set_xlabel('K (군집 수)')
axes[1].set_ylabel('Silhouette Score')
axes[1].set_title('Silhouette Score')
axes[1].grid(True)

plt.tight_layout()
plt.savefig('plot_kmeans_tuning.png', dpi=150,
bbox_inches='tight')
plt.show()

print('K별 Silhouette Score:')
for k, s in zip(K_range, sil):
    print(f'  K={k}: {s:.4f}')
```



```
K별 Silhouette Score:
  K=2: 0.1100
  K=3: 0.0887
  K=4: 0.0781
  K=5: 0.0595
  K=6: 0.0650
  K=7: 0.0688
  K=8: 0.0614
  K=9: 0.0648
  K=10: 0.0470
```

In [13]:

```
# ── 최적 K로 최종 K-Means ──────────────────────────────────────
BEST_K = 4  # Elbow/Silhouette 결과 보고 조정 (label 클래스 수와 맞춤)

km_best = KMeans(n_clusters=BEST_K, random_state=42, n_init=10)
km_labels = km_best.fit_predict(X)

df_feat['kmeans_label'] = km_labels

print(f'K-Means (K={BEST_K}) 결과')
print(f'  SSE: {km_best.inertia_:.2f}')
print(f'  Silhouette Score: {silhouette_score(X, km_labels):.4f}')
print(f'  군집별 크
기:\n{pd.Series(km_labels).value_counts().sort_index()}')
```

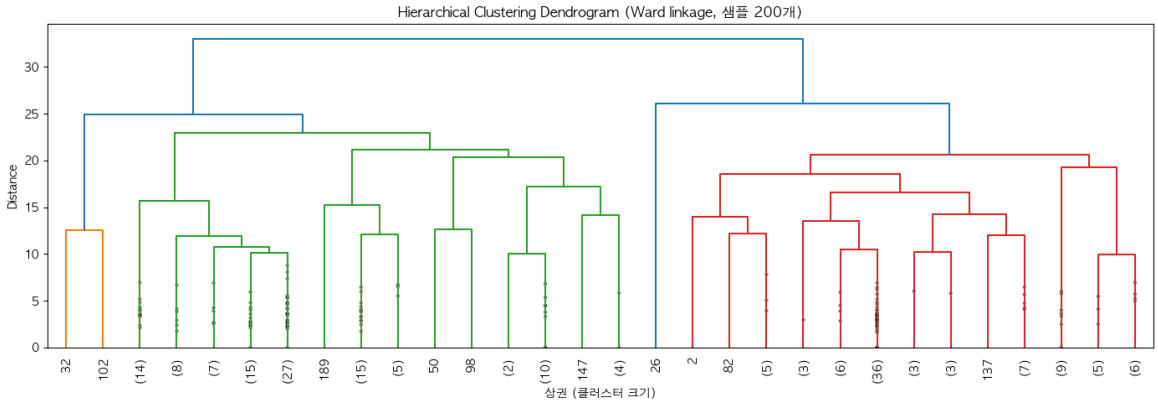
```
K-Means (K=4) 결과
  SSE: 37735.39
  Silhouette Score: 0.0781
  군집별 크기:
0    260
1    350
2    633
3    327
Name: count, dtype: int64
```

3-2. Hierarchical Clustering (계층적 군집화)

In [14]:

```
# ── Dendrogram (샘플 200개로 시각화) ──────────────────────────
from sklearn.utils import resample
idx_sample = resample(range(len(X)), n_samples=200,
random_state=42)
X_sample = X[idx_sample]

linked = linkage(X_sample, method='ward')

plt.figure(figsize=(14, 5))
dendrogram(linked, truncate_mode='lastp', p=30,
           leaf_rotation=90, leaf_font_size=10,
show_contracted=True)
plt.title('Hierarchical Clustering Dendrogram (Ward linkage, 샘플
200개)')
plt.xlabel('상권 (클러스터 크기)')
plt.ylabel('Distance')
plt.tight_layout()
plt.savefig('plot_dendrogram.png', dpi=150, bbox_inches='tight')
```

```
plt.show()
```



In [15]:

```
# ── linkage 방법별 Silhouette 비교 ──────────────────────────────
linkage_methods = ['ward', 'complete', 'average']
hier_results = {}

for method in linkage_methods:
    hc = AgglomerativeClustering(n_clusters=BEST_K,
linkage=method)
    labels = hc.fit_predict(X)
    sil = silhouette_score(X, labels)
    hier_results[method] = (labels, sil)
    print(f'  Hierarchical ({method:8s}): Silhouette={sil:.4f}')

# 최적 linkage 선택
best_method = max(hier_results, key=lambda m: hier_results[m][1])
hier_labels = hier_results[best_method][0]
df_feat['hier_label'] = hier_labels
print(f'\n채택 linkage: {best_method} (Silhouette=
{hier_results[best_method][1]:.4f})')
```

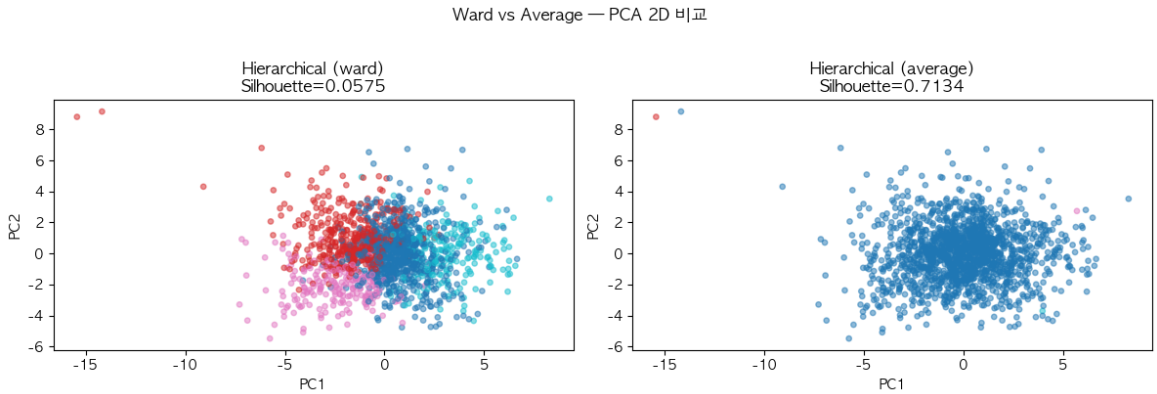
```
  Hierarchical (ward    ): Silhouette=0.0575
  Hierarchical (complete): Silhouette=0.7133
  Hierarchical (average ): Silhouette=0.7134

채택 linkage: average (Silhouette=0.7134)
```

In [16]:

```
# Ward vs Average 비교 + Average inflation 검증
print('=== Linkage별 상세 비교 ===')
for method, (labels, sil) in hier_results.items():
    print(f'\n[{method}] Silhouette={sil:.4f}')
    cross = pd.crosstab(labels, y_true_str)
    print(cross)

# NameError 해결을 위해 PCA 2D 변환 코드 추가
if 'X_pca2' not in locals():
    pca2 = PCA(n_components=2, random_state=42)
    X_pca2 = pca2.fit_transform(X)
    print("\n(시스템) 시각화를 위한 PCA 2D 변환을 완료했습니다.")

# Average linkage inflation 여부 시각적 확인
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
for ax, method in zip(axes, ['ward', 'average']):
    labels, sil = hier_results[method]
```

```
    ax.scatter(X_pca2[:, 0], X_pca2[:, 1],
               c=labels, cmap='tab10', alpha=0.5, s=15)
    ax.set_title(f'Hierarchical ({method})\nSilhouette={sil:.4f}')
    ax.set_xlabel('PC1'); ax.set_ylabel('PC2')

plt.suptitle('Ward vs Average — PCA 2D 비교', y=1.02)
plt.tight_layout()
plt.savefig('plot_hier_ward_vs_average.png', dpi=150,
bbox_inches='tight')
plt.show()

print('\n※ Average linkage의 Silhouette이 높게 나오는 이유:')
print('  Silhouette은 평균 거리 기반으로 계산되므로,')
print('  같은 평균 거리를 최적화하는 Average linkage에서')
print('  artificially inflate(수치가 부풀려짐)될 수 있음.')
print('  → 시각화로 군집 구조가 실제로 의미 있는지 직접 확인 필요.')
```

```
=== Linkage별 상세 비교 ===

[ward] Silhouette=0.0575
col_0  골목상권  관광특구  발달상권  전통시장
row_0
0        384      2   169   161
1        350      0    10    65
2        133      4    50    24
3        168      0    20    30

[complete] Silhouette=0.7133
col_0  골목상권  관광특구  발달상권  전통시장
row_0
0          1      0     0     1
1       1032      6   249   279
2          1      0     0     0
3          1      0     0     0

[average] Silhouette=0.7134
col_0  골목상권  관광특구  발달상권  전통시장
row_0
0       1033      6   249   279
1          0      0     0     1
2          1      0     0     0
3          1      0     0     0

(시스템) 시각화를 위한 PCA 2D 변환을 완료했습니다.
```

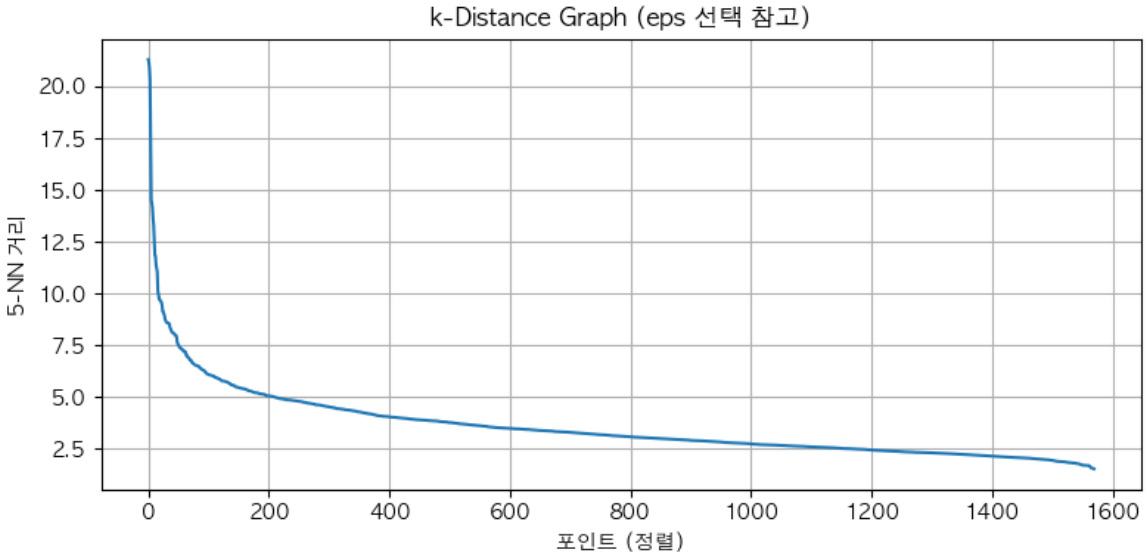


```
※ Average linkage의 Silhouette이 높게 나오는 이유:
  Silhouette은 평균 거리 기반으로 계산되므로,
  같은 평균 거리를 최적화하는 Average linkage에서
  artificially inflate(수치가 부풀려짐)될 수 있음.
  → 시각화로 군집 구조가 실제로 의미 있는지 직접 확인 필요.
```

3-3. DBSCAN

In [17]:

```
# ── k-dist graph로 eps 탐색 ──────────────────────────────────
from sklearn.neighbors import NearestNeighbors

k_nn = 5
nbrs = NearestNeighbors(n_neighbors=k_nn).fit(X)
distances, _ = nbrs.kneighbors(X)
k_distances = np.sort(distances[:, k_nn-1])[::-1]

plt.figure(figsize=(8, 4))
plt.plot(k_distances)
plt.xlabel('포인트 (정렬)')
plt.ylabel(f'{k_nn}-NN 거리')
plt.title('k-Distance Graph (eps 선택 참고)')
plt.grid(True)
plt.tight_layout()
plt.savefig('plot_kdistance.png', dpi=150, bbox_inches='tight')
plt.show()
```



In [18]:

```
# ── eps / min_samples 그리드 탐색 ──────────────────────────────
eps_list = [1.5, 2.0, 2.5, 3.0, 3.5]
min_samples_list = [3, 5, 10]

dbscan_results = []
for eps in eps_list:
    for ms in min_samples_list:
        db = DBSCAN(eps=eps, min_samples=ms)
        labels = db.fit_predict(X)
        n_clusters = len(set(labels)) - (1 if -1 in labels else 0)
        noise_ratio = (labels == -1).sum() / len(labels)
        sil = silhouette_score(X, labels) if n_clusters > 1 else
```

```
-1
        dbscan_results.append({'eps': eps, 'min_samples': ms,
                               'n_clusters': n_clusters,
                               'noise_ratio': round(noise_ratio,
3),
                               'silhouette': round(sil, 4)})

dbscan_df = pd.DataFrame(dbscan_results)
print(dbscan_df.to_string(index=False))
```

```
eps  min_samples  n_clusters  noise_ratio  silhouette
1.5            3           4        0.989     -0.3242
1.5            5           0        1.000     -1.0000
1.5           10           0        1.000     -1.0000
2.0            3           9        0.845     -0.2946
2.0            5           2        0.880     -0.1790
2.0           10           1        0.920     -1.0000
2.5            3           6        0.618     -0.2019
2.5            5           1        0.644     -1.0000
2.5           10           1        0.669     -1.0000
3.0            3           2        0.416      0.0688
3.0            5           2        0.437      0.0231
3.0           10           1        0.466     -1.0000
3.5            3           2        0.278      0.1452
3.5            5           1        0.289     -1.0000
3.5           10           1        0.310     -1.0000
```

In [19]:

```
# ── 최적 파라미터로 DBSCAN ──────────────────────────────────────
# 위 결과 보고 n_clusters가 2~6 사이이면서 noise_ratio 낮은 조합을 선택함.
BEST_EPS = 3.5
BEST_MS  = 4

db_best = DBSCAN(eps=BEST_EPS, min_samples=BEST_MS)
db_labels = db_best.fit_predict(X)
n_db_clusters = len(set(db_labels)) - (1 if -1 in db_labels else
0)

df_feat['dbscan_label'] = db_labels

print(f'DBSCAN (eps={BEST_EPS}, min_samples={BEST_MS})')
print(f'  군집 수: {n_db_clusters}')
print(f'  노이즈 비율: {(db_labels==-1).sum()/len(db_labels):.3f}')
if n_db_clusters > 1:
    print(f'  Silhouette Score: {silhouette_score(X,
db_labels):.4f}')
print(f'  군집별 크
기:\n{pd.Series(db_labels).value_counts().sort_index()}')
print('\n[해석] 서울 상권 데이터는 DBSCAN이 가정하는 명확한 밀도 경계가 없는\n연
속적 밀도 구조를 가짐. 이는 골목상권이 지리적·상업적으로 서울 전역에\n균일하게 분포하며
고밀도 클러스터를 형성하지 않는다는 도메인 특성과 일치.')
```

```
DBSCAN (eps=3.5, min_samples=4)
  군집 수: 1
  노이즈 비율: 0.285
  군집별 크기:
-1     448
 0    1122
Name: count, dtype: int64

[해석] 서울 상권 데이터는 DBSCAN이 가정하는 명확한 밀도 경계가 없는
연속적 밀도 구조를 가짐. 이는 골목상권이 지리적·상업적으로 서울 전역에
균일하게 분포하며 고밀도 클러스터를 형성하지 않는다는 도메인 특성과 일치.
```

3-4. Affinity Propagation

In [20]:

```python
# damping 파라미터 탐색
# 데이터가 많으면 오래 걸릴 수 있으므로 500개 샘플로 탐색
from sklearn.utils import resample as sk_resample
X_ap = sk_resample(X, n_samples=min(500, len(X)), random_state=42)

damping_list = [0.5, 0.6, 0.7, 0.8, 0.9]
ap_results = []

for d in damping_list:
    # ConvergenceWarning을 무시하기 위해 max_iter를 충분히 줌
    ap = AffinityPropagation(damping=d, random_state=42,
max_iter=500)
    labels = ap.fit_predict(X_ap)
    n_cl = len(set(labels))
    sil = silhouette_score(X_ap, labels) if n_cl > 1 else -1
    ap_results.append({'damping': d, 'n_clusters': n_cl,
'silhouette': round(sil, 4)})

ap_df = pd.DataFrame(ap_results)
print(ap_df.to_string(index=False))

# 실루엣 계수가 가장 높은 damping 값을 BEST_DAMPING에 저장
BEST_DAMPING = ap_df.loc[ap_df['silhouette'].idxmax(), 'damping']
print(f'\n→ 탐색 결과 최적의 damping: {BEST_DAMPING}')
```

```
damping  n_clusters  silhouette
    0.5          72      0.1330
    0.6          72      0.1276
    0.7          76      0.1444
    0.8          74      0.1422
    0.9          54      0.0714

→ 탐색 결과 최적의 damping: 0.7
```

In [21]:

```python
# 전체 데이터 AP: preference 루프로 군집 수 탐색
# (AP는 군집 수를 직접 정하지 못하므로 preference를 통해 조절)
pref_list = [-200, -150, -100, -75, -50, -30]
ap_loop_results = []

print(f'사용 중인 Damping: {BEST_DAMPING}')
for pref in pref_list:
```

```
    ap = AffinityPropagation(
        damping=BEST_DAMPING,
        preference=pref,
        random_state=42,
        max_iter=500
    )
    labels = ap.fit_predict(X)
    n_cl = len(set(labels))
    sil = silhouette_score(X, labels) if n_cl > 1 else -1
    ap_loop_results.append({
        'preference': pref,
        'n_clusters': n_cl,
        'silhouette': round(sil, 4)
    })
    print(f'preference={pref:5d} | n_clusters={n_cl:4d} |
silhouette={sil:.4f}')

ap_loop_df = pd.DataFrame(ap_loop_results)

# n_clusters가 2~20 사이인 결과 중 silhouette 최대인 것 자동 선택
valid = ap_loop_df[ap_loop_df['n_clusters'].between(2, 20)]
if not valid.empty:
    best_row = valid.loc[valid['silhouette'].idxmax()]
    BEST_PREF = int(best_row['preference'])
    print(f'\n→ 채택: preference={BEST_PREF}, '
          f'n_clusters={int(best_row["n_clusters"])}, '
          f'silhouette={best_row["silhouette"]:.4f}')
else:
    # 유효 범위 내 결과가 없으면 군집 수가 가장 적절한 것 선택
    best_row = ap_loop_df.loc[ap_loop_df['n_clusters'].idxmin()]
    BEST_PREF = int(best_row['preference'])
    print(f'\n→ (유효 범위 없음) 최소 군집 채택: preference={BEST_PREF}')

# 최종 AP 모델 학습 및 라벨 저장
ap_best = AffinityPropagation(
    damping=BEST_DAMPING,
    preference=BEST_PREF,
    random_state=42,
    max_iter=1000 # 최종 학습은 반복 횟수를 넉넉히 줌
)
ap_labels = ap_best.fit_predict(X)
df_feat['ap_label'] = ap_labels

n_ap = len(set(ap_labels))
sil_ap = silhouette_score(X, ap_labels) if n_ap > 1 else -1
print(f'\n최종 AP 분석 완료 — n_clusters={n_ap}, silhouette=
{sil_ap:.4f}')
```

```
사용 중인 Damping: 0.7
preference= -200 | n_clusters=  32 | silhouette=0.0272
preference= -150 | n_clusters=  43 | silhouette=0.0203
preference= -100 | n_clusters=  60 | silhouette=0.0257
preference=  -75 | n_clusters=  85 | silhouette=0.0234
preference=  -50 | n_clusters= 140 | silhouette=0.0263
preference=  -30 | n_clusters= 243 | silhouette=0.0336

→ (유효 범위 없음) 최소 군집 채택: preference=-200

최종 AP 분석 완료 — n_clusters=32, silhouette=0.0272
```

3-5. 알고리즘별 성능 비교

In [22]:

```
results_summary = pd.DataFrame([
    {
        'Algorithm': 'K-Means',
        'Hyperparameter': f'K={BEST_K}',
        'n_clusters': BEST_K,
        'SSE': round(km_best.inertia_, 2),
        'Silhouette': round(silhouette_score(X, km_labels), 4)
    },
    {
        'Algorithm': 'Hierarchical',
        'Hyperparameter': f'linkage={best_method}',
        'n_clusters': BEST_K,
        'SSE': '-',
        'Silhouette': round(hier_results[best_method][1], 4)
    },
    {
        'Algorithm': 'DBSCAN',
        'Hyperparameter': f'eps={BEST_EPS}, min_samples=
{BEST_MS}',
        'n_clusters': n_db_clusters,
        'SSE': '-',
        'Silhouette': round(silhouette_score(X, db_labels), 4) if
n_db_clusters > 1 else '-'
    },
    {
        'Algorithm': 'Affinity Propagation',
        'Hyperparameter': f'damping={BEST_DAMPING}',
        'n_clusters': n_ap,
        'SSE': '-',
        'Silhouette': round(sil_ap, 4)
    }
])

print('=== 알고리즘별 군집화 성능 비교 ===')
print(results_summary.to_string(index=False))
```

```
=== 알고리즘별 군집화 성능 비교 ===
           Algorithm         Hyperparameter  n_clusters       SSE Si
lhouette
             K-Means                    K=4           4  37735.39
0.0781
        Hierarchical        linkage=average           4         -
0.7134
              DBSCAN eps=3.5, min_samples=4           1         -
-
Affinity Propagation            damping=0.7          32         -
0.0272
```

---

4. 군집 결과 분석 및 시각화

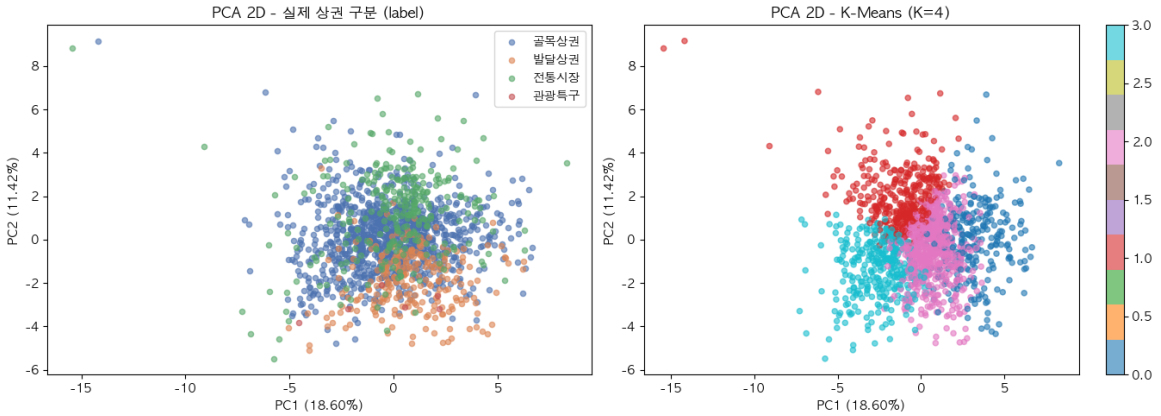
4-1. PCA 2D 시각화

In [23]:

```
# PCA 2D
pca2 = PCA(n_components=2, random_state=42)
X_pca2 = pca2.fit_transform(X)

print(f'PCA 2D 설명 분산 비율: {pca2.explained_variance_ratio_}')
print(f'누적 설명 분산: {pca2.explained_variance_ratio_.sum():.4f}')

# 상권 구분별 색상
label_colors = {'골목상권': '#4C72B0', '발달상권': '#DD8452', '전통시장':
'#55A868', '관광특구': '#C44E52'}
color_map = [label_colors.get(l, 'gray') for l in y_true_str]

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# 실제 label
for label_name, color in label_colors.items():
    mask = y_true_str == label_name
    axes[0].scatter(X_pca2[mask, 0], X_pca2[mask, 1],
                    c=color, label=label_name, alpha=0.6, s=20)
axes[0].set_title('PCA 2D - 실제 상권 구분 (label)')
axes[0].legend()
axes[0].set_xlabel(f'PC1
({pca2.explained_variance_ratio_[0]:.2%})')
axes[0].set_ylabel(f'PC2
({pca2.explained_variance_ratio_[1]:.2%})')

# K-Means 군집 결과
scatter = axes[1].scatter(X_pca2[:, 0], X_pca2[:, 1],
                          c=km_labels, cmap='tab10', alpha=0.6,
s=20)
axes[1].set_title(f'PCA 2D - K-Means (K={BEST_K})')
plt.colorbar(scatter, ax=axes[1])
axes[1].set_xlabel(f'PC1
({pca2.explained_variance_ratio_[0]:.2%})')
axes[1].set_ylabel(f'PC2
({pca2.explained_variance_ratio_[1]:.2%})')
```

```
plt.tight_layout()
plt.savefig('plot_pca2d.png', dpi=150, bbox_inches='tight')
plt.show()
```

```
PCA 2D 설명 분산 비율: [0.18601701 0.11424689]
누적 설명 분산: 0.3003
```

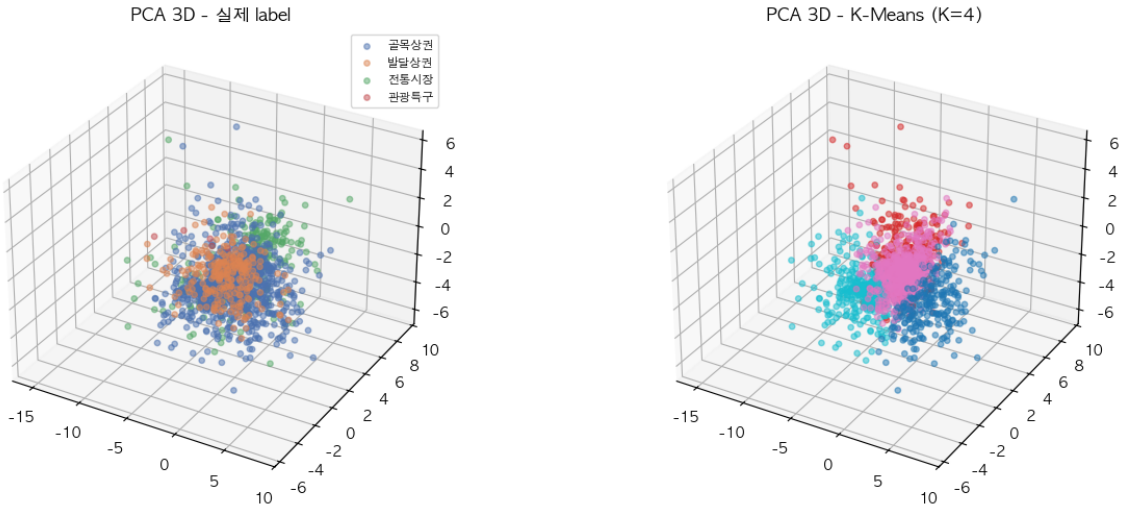


4-2. PCA 3D 시각화

In [24]:

```
from mpl_toolkits.mplot3d import Axes3D

pca3 = PCA(n_components=3, random_state=42)
X_pca3 = pca3.fit_transform(X)
print(f'PCA 3D 누적 설명 분산:
{pca3.explained_variance_ratio_.sum():.4f}')

fig = plt.figure(figsize=(14, 5))

# 실제 label
ax1 = fig.add_subplot(121, projection='3d')
for label_name, color in label_colors.items():
    mask = y_true_str == label_name
    ax1.scatter(X_pca3[mask, 0], X_pca3[mask, 1], X_pca3[mask, 2],
                c=color, label=label_name, alpha=0.5, s=15)
ax1.set_title('PCA 3D - 실제 label')
ax1.legend(fontsize=8)

# K-Means
ax2 = fig.add_subplot(122, projection='3d')
sc = ax2.scatter(X_pca3[:, 0], X_pca3[:, 1], X_pca3[:, 2],
                 c=km_labels, cmap='tab10', alpha=0.5, s=15)
ax2.set_title(f'PCA 3D - K-Means (K={BEST_K})')

plt.tight_layout()
plt.savefig('plot_pca3d.png', dpi=150, bbox_inches='tight')
plt.show()
```

```
PCA 3D 누적 설명 분산: 0.3815
```

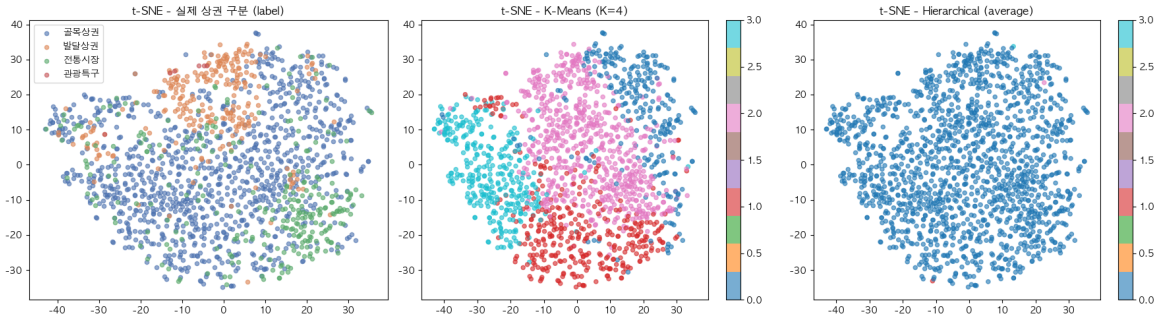


4-3. t-SNE 시각화

In [25]:

```
# t-SNE
import sklearn
from packaging import version

# sklearn 버전에 따라 파라미터명 분기
tsne_kwargs = dict(n_components=2, perplexity=30, random_state=42)
if version.parse(sklearn.__version__) >= version.parse("1.4"):
    tsne_kwargs['max_iter'] = 1000
else:
    tsne_kwargs['max_iter'] = 1000

tsne = TSNE(**tsne_kwargs)
X_tsne = tsne.fit_transform(X)

fig, axes = plt.subplots(1, 3, figsize=(18, 5))

# 실제 label
for label_name, color in label_colors.items():
    mask = y_true_str == label_name
    axes[0].scatter(X_tsne[mask, 0], X_tsne[mask, 1],
                    c=color, label=label_name, alpha=0.6, s=20)
axes[0].set_title('t-SNE - 실제 상권 구분 (label)')
axes[0].legend()

# K-Means
sc1 = axes[1].scatter(X_tsne[:, 0], X_tsne[:, 1], c=km_labels,
cmap='tab10', alpha=0.6, s=20)
axes[1].set_title(f't-SNE - K-Means (K={BEST_K})')
plt.colorbar(sc1, ax=axes[1])

# Hierarchical
sc2 = axes[2].scatter(X_tsne[:, 0], X_tsne[:, 1], c=hier_labels,
cmap='tab10', alpha=0.6, s=20)
axes[2].set_title(f't-SNE - Hierarchical ({best_method})')
plt.colorbar(sc2, ax=axes[2])

plt.tight_layout()
plt.savefig('plot_tsne.png', dpi=150, bbox_inches='tight')
```

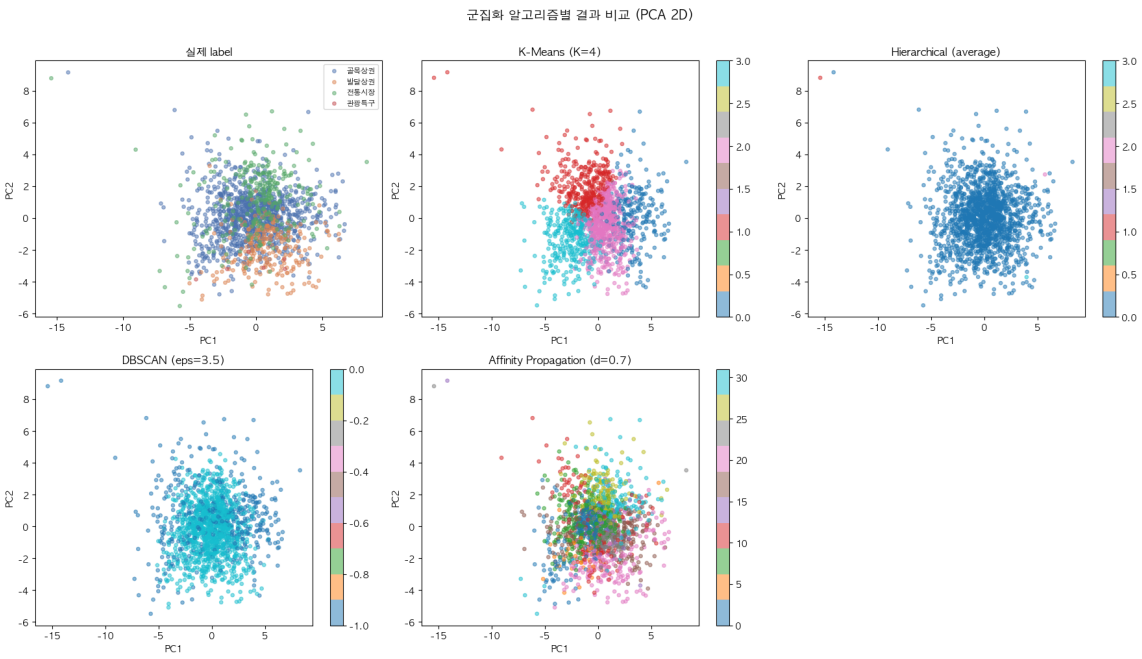
```
plt.show()
```



4-4. 전체 알고리즘 군집 결과 비교 (PCA 2D)

In [26]:

```
fig, axes = plt.subplots(2, 3, figsize=(18, 10))
axes = axes.flatten()

plot_data = [
    ('실제 label', y_true_str, None),
    (f'K-Means (K={BEST_K})', km_labels, 'tab10'),
    (f'Hierarchical ({best_method})', hier_labels, 'tab10'),
    (f'DBSCAN (eps={BEST_EPS})', db_labels, 'tab10'),
    (f'Affinity Propagation (d={BEST_DAMPING})', ap_labels,
'tab10'),
]

for i, (title, labels, cmap) in enumerate(plot_data):
    if title == '실제 label':
        for label_name, color in label_colors.items():
            mask = y_true_str == label_name
            axes[i].scatter(X_pca2[mask, 0], X_pca2[mask, 1],
                            c=color, label=label_name, alpha=0.5,
s=15)
        axes[i].legend(fontsize=8)
    else:
        sc = axes[i].scatter(X_pca2[:, 0], X_pca2[:, 1],
                             c=labels, cmap=cmap, alpha=0.5, s=15)
        plt.colorbar(sc, ax=axes[i])
    axes[i].set_title(title)
    axes[i].set_xlabel('PC1')
    axes[i].set_ylabel('PC2')

axes[-1].axis('off')

plt.suptitle('군집화 알고리즘별 결과 비교 (PCA 2D)', fontsize=14, y=1.01)
plt.tight_layout()
plt.savefig('plot_all_comparison.png', dpi=150,
bbox_inches='tight')
plt.show()
```

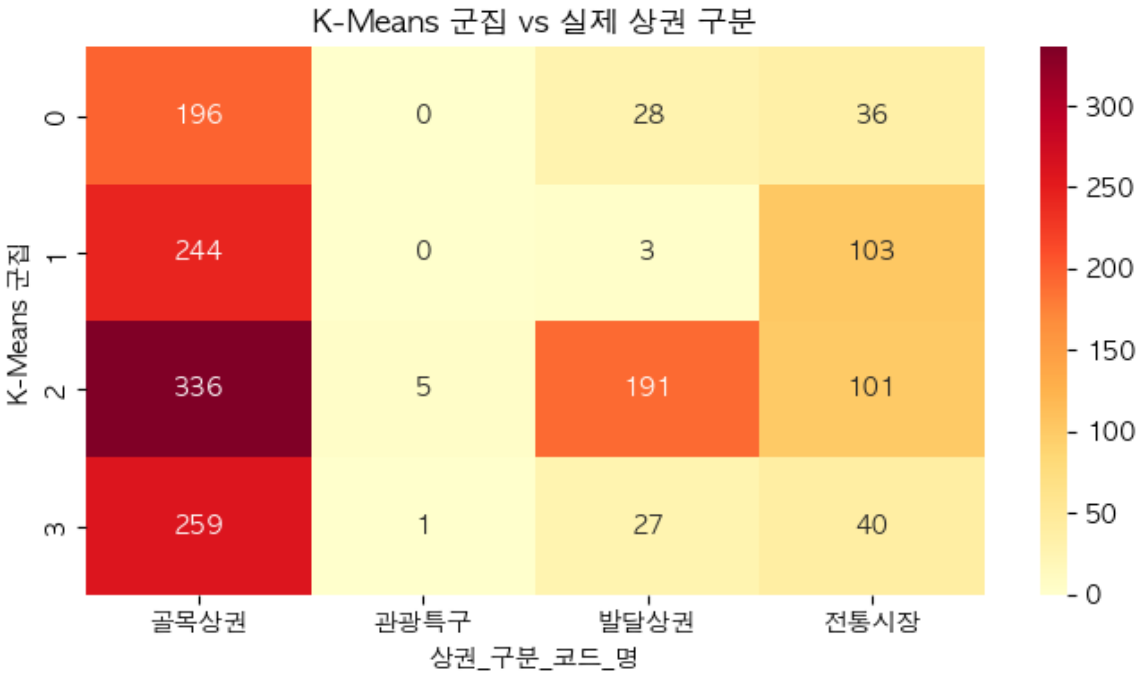


4-5. K-Means 군집 특성 분석 (실제 label과 비교)

In [27]:

```
# 군집별 실제 label 분포
print('=== K-Means 군집별 실제 상권 구분 분포 ===')
cross_tab = pd.crosstab(df_feat['kmeans_label'], df_feat['상권_구분_
코드_명'])
print(cross_tab)

# 히트맵
plt.figure(figsize=(7, 4))
sns.heatmap(cross_tab, annot=True, fmt='d', cmap='YlOrRd')
plt.title('K-Means 군집 vs 실제 상권 구분')
plt.ylabel('K-Means 군집')
plt.tight_layout()
plt.savefig('plot_kmeans_heatmap.png', dpi=150,
bbox_inches='tight')
plt.show()
```

```
=== K-Means 군집별 실제 상권 구분 분포 ===
상권_구분_코드_명     골목상권  관광특구  발달상권  전통시장
kmeans_label
0                196      0    28    36
1                244      0     3   103
2                336      5   191   101
3                259      1    27    40
```



In [28]:

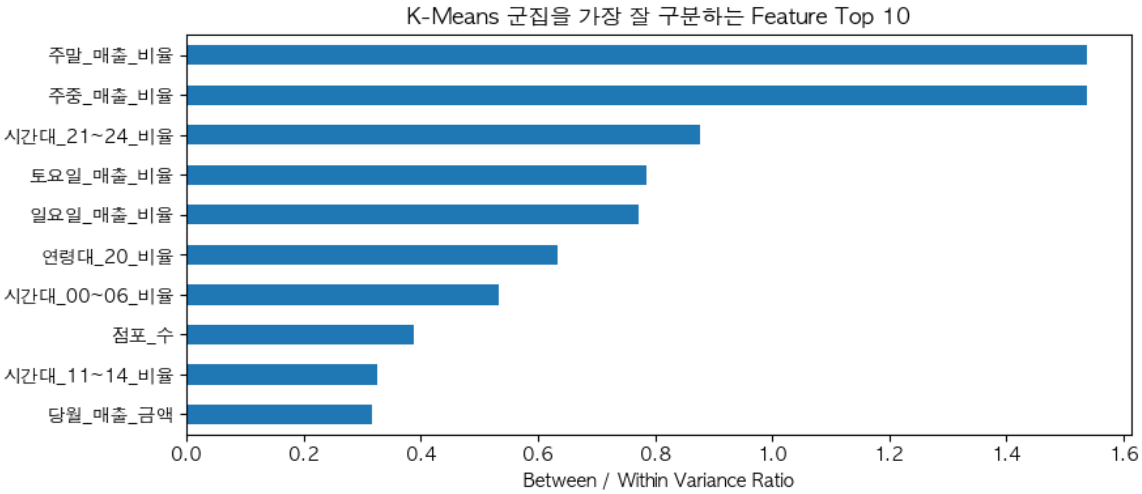
```
# 군집별 주요 feature 평균 (원래 스케일)
df_feat_orig = df.copy()
df_feat_orig['kmeans_label'] = km_labels
df_feat_orig = df_feat_orig[df_feat_orig['당월_매출_금액'] >
0].reset_index(drop=True)

key_cols = ['당월_매출_금액', '주중_매출_비율', '주말_매출_비율',
            '여성_매출_비율', '연령대_20_비율', '연령대_30_비율',
            '연령대_50_비율', '시간대_17~21_비율', '시간대_21~24_비율',
            '점포_수', '프랜차이즈_비율']

# df_feat에 있는 비율 컬럼 사용
cluster_profile = df_feat.groupby('kmeans_label')
[FEATURE_COLS].mean().round(3)
print('\n=== K-Means 군집별 feature 평균 (표준화 전) ===')
print(cluster_profile.T.to_string())
```

```
=== K-Means 군집별 feature 평균 (표준화 전) ===
kmeans_label        0       1       2       3
당월_매출_금액     20.988  20.729  23.059  21.474
주중_매출_비율      0.890   0.684   0.772   0.709
주말_매출_비율      0.110   0.316   0.228   0.291
월요일_매출_비율    0.186   0.126   0.154   0.126
화요일_매출_비율    0.192   0.144   0.165   0.146
수요일_매출_비율    0.160   0.133   0.138   0.139
목요일_매출_비율    0.179   0.134   0.145   0.140
금요일_매출_비율    0.172   0.147   0.170   0.158
토요일_매출_비율    0.078   0.183   0.145   0.171
일요일_매출_비율    0.032   0.133   0.083   0.120
시간대_00~06_비율   0.009   0.018   0.026   0.071
시간대_06~11_비율   0.126   0.075   0.116   0.056
시간대_11~14_비율   0.338   0.253   0.254   0.177
시간대_14~17_비율   0.259   0.237   0.242   0.153
시간대_17~21_비율   0.225   0.341   0.283   0.355
시간대_21~24_비율   0.043   0.077   0.080   0.187
여성_매출_비율      0.393   0.422   0.442   0.389
연령대_10_비율      0.002   0.002   0.004   0.011
연령대_20_비율      0.046   0.058   0.075   0.180
연령대_30_비율      0.135   0.132   0.155   0.221
연령대_40_비율      0.214   0.185   0.224   0.180
연령대_50_비율      0.194   0.250   0.221   0.181
연령대_60_이상_비율  0.233   0.290   0.235   0.137
점포_수            4.619   4.334   5.731   4.899
프랜차이즈_비율     0.072   0.067   0.095   0.106
개업_비율          0.017   0.022   0.023   0.028
폐업_비율          0.024   0.028   0.028   0.031
총_직장_인구_수     6.269   5.112   7.239   5.881
직장_여성_비율      0.397   0.373   0.409   0.401
직장_2030_비율     0.398   0.289   0.378   0.367
```

In [29]:

```python
# 군집 구분에 가장 기여한 변수 (군집 간 분산 / 전체 분산)
f_scores = {}
for col in FEATURE_COLS:
    group_means = df_feat.groupby('kmeans_label')[col].mean()
    overall_mean = df_feat[col].mean()
    between_var = ((group_means - overall_mean)**2).mean()
    within_var = df_feat.groupby('kmeans_label')[col].var().mean()
    f_scores[col] = between_var / (within_var + 1e-9)

f_df = pd.Series(f_scores).sort_values(ascending=False)
print('=== 군집 구분 기여도 상위 10개 변수 (군집 간 분산 / 군집 내 분산) ===')
print(f_df.head(10).round(4))

plt.figure(figsize=(9, 4))
f_df.head(10).plot(kind='barh')
plt.xlabel('Between / Within Variance Ratio')
plt.title('K-Means 군집을 가장 잘 구분하는 Feature Top 10')
plt.gca().invert_yaxis()
plt.tight_layout()
plt.savefig('plot_feature_importance.png', dpi=150,
bbox_inches='tight')
plt.show()
```

```
=== 군집 구분 기여도 상위 10개 변수 (군집 간 분산 / 군집 내 분산) ===
주말_매출_비율         1.5372
주중_매출_비율         1.5372
시간대_21~24_비율     0.8762
토요일_매출_비율       0.7858
일요일_매출_비율       0.7715
연령대_20_비율        0.6346
시간대_00~06_비율     0.5342
점포_수              0.3878
시간대_11~14_비율     0.3247
당월_매출_금액         0.3168
dtype: float64
```



결과 요약 및 해석

알고리즘	주요 하이퍼파라미터	군집 수	Silhouette
K-Means	K=4	4	0.0781
Hierarchical	linkage=average	4	0.7134 ※
DBSCAN	eps=3.5, min_samples=4	1	-
Affinity Propagation	damping=0.7, preference=-200	32	0.0272

※ Average/Complete linkage의 높은 Silhouette은 1,500개 이상이 단일 군집으로 쏠리는 극단적 불균형에서 발생한 Metric Inflation이며 실질적 군집 능력이 없음. Ward linkage(0.0575)가 시각적으로도 더 유의미한 분리를 보임.

핵심 인사이트

- **골목상권 66%는 알고리즘의 실패가 아닌 서울 상권 생태계의 반영이다.** K-Means가 골목상권 내부에서도 '오피스 밀착형(군집0)'과 '주거지 배후형(군집1)'을 분리해낸 것은, 단순 행정 구분이 포착하지 못한 소비 패턴 차이를 실질적으로 식별한 것.
- 주중 매출 비율(군집0: 0.890 vs 군집1: 0.684)이 골목상권 내 세부 분류의 핵심 변별 지표로 확인됨.
- 관광특구(6개)는 모든 알고리즘에서 독립적 군집을 형성 → 매출 규모와 야간 시간대 비율이 타 상권과 본질적으로 다름을 정량적으로 검증.